

Relatório 2 - Aplicações em visão computacional (I)

Felipe Fonseca

Descrição da atividade

Esse Card contempla um vídeo e um blog que demonstram um fluxo de como utilizar a ferramenta da Nvidia para geração de dados sintéticos. O vídeo faz uma demonstração de como são os ambientes criados pela ferramenta da Nvidia e mostra um pouco da prática, sendo um vídeo bem curto, servindo mais para marketplace do que para explicar algo de fato.

Já o blog inicia já falando sobre os dados sintéticos no treinamento de modelos multimodais, onde eles citam o Nvidia Omniverse Replicator para criar um pipeline personalizado para isso. Em muitos momentos ficam citando as vantagens de se utilizar dados sintéticos, mas como isso foi conteúdo do Card anterior, irei focar realmente no que diz respeito ao Nvidia Omniverse.

O Omniverse Replicator é uma ferramenta que está disponível dentro do SDK do Nvidia Omniverse, que serve para a geração de dados sintéticos 3D, permitindo assim que cenas reais possam ser replicadas dentro do ambiente, contendo informações importantes como iluminação, profundidade, altura, distância etc. Esse tipo de cena 3D pode ser utilizado para treinar modelos de carros autônomos, robôs etc.

Dentre as vantagens do Omniverse replicator, tem a capacidade de customizar todo o ambiente 3D como quiser, além de ser fácil de adicionar metadados para qualquer cena (pode ser programável). Mas as principais vantagens que eu considero mais importante são: suporte a múltiplas GPUs para renderização do ambiente, o que é fundamental para o funcionamento em servidores e data centers, e o suporte a importação de arquivos de outras plataformas de criação de ambientes 3D, como CAD e principalmente o Blender. A ferramenta também permite a aleatoriedade na geração dos cenários, incluindo iluminação, posicionamento dos objetos, cores etc.

Outra vantagem do replicator, é a capacidade de poder ser controlado por scripts em python, podendo gerar automatizações para gerar cenários mais facilmente, sendo esses cenários bem mais dinâmicos e complexos.

Um ponto interessante são os desafios de cenários 3D que normalmente são encontrados:

- Appearance Gap: a disparidade de aparência é a diferença dos objetos sintéticos dos objetos reais a nível de pixels, isto é, quando um objeto é criado sinteticamente, ele tende a ser sempre um objeto mais lizo, com bordas perfeitas, ou sem iluminação realista, o que pode acabar interferindo no treinamento de um modelo;
- Content Gap: já a disparidade de conteúdo, nada mais é do que a diferença na composição da cena, ou seja, se os dados sintéticos criam sempre objetos alinhados em perfeito estados, o modelo pode falhar ao reconhecer esses mesmos objetos em cenários reais.

Entendendo então como esse framework da Nvidia pode resolver esses problemas, a conclusão foi:

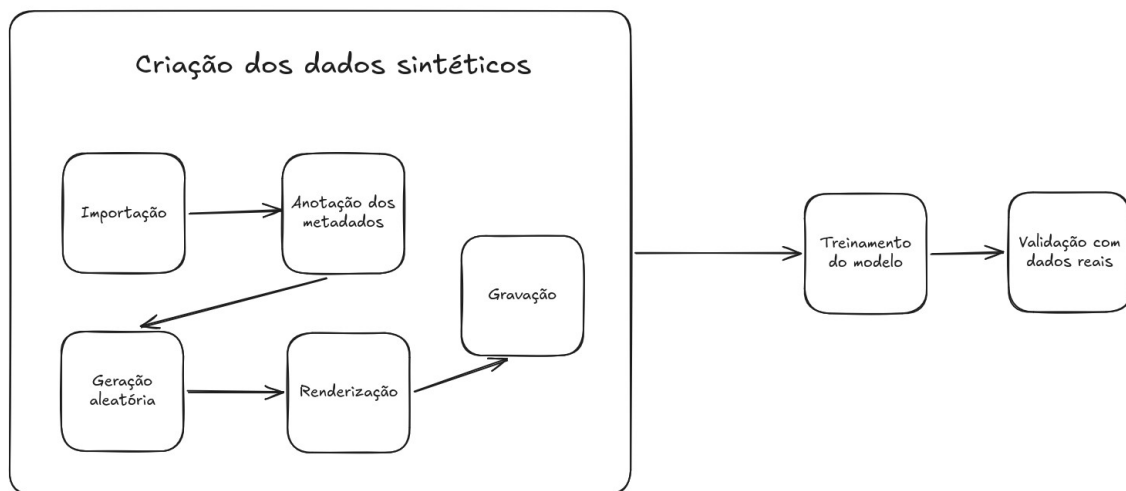
- O appearance gap é resolvido através da renderização que utiliza path tracing e ray tracing para simular o comportamento das sombras e reflexos, além de permitir a criação de propriedades físicas dos objetos, garantindo que as superfícies dos objetos pareçam iguais na vida real;
- O content gap é resolvido através da aleatoriedade do Domain Randomization via os scripts em python, que permite criar múltiplos cenários com o objetivo de cobrir tudo que for importante, podendo até gerar casos extremos que seriam difíceis de ver na vida real, como cenários de riscos ou que contenham informações privadas.

Abaixo, uma imagem retirada do site “macnica” que demonstra diferentes tipos de demonstrações do mesmo cenário gerado através do Replicator:



Projetando um possível fluxo de desenvolvimento com o Nvidia Replicator, eu desenhei esse pipeline:

Pipeline do Omniverse Replicator



- Importação: a primeira fase seria a de importar os assets 3D que irão formar o cenário, podendo ser o background, objetos 3D que serão detectados, texturas etc;
- Anotação dos metadados: a segunda fase seria anotar as informações de classes na hierarquia dos objetos, para que o render saiba o que cada objeto representa;

- Geração aleatória: a terceira fase é a definição das regras para a geração dos cenários de forma aleatória;
- Renderização: a quarta fase é a parte de associar quais dados você deseja extrair na renderização, por exemplo, se irá querer apenas o cenário em RGB ou em HDR, coisas desse tipo;
- Gravação: a finalização é apenas rodar a ferramenta para que seja feita a geração dos cenários.

Quanto aos cuidados a serem tomados, o principal que eu determinei no pipeline, é que após o treinamento do modelo com os dados sintéticos, a validação deve ser feita com dados reais, pois é a forma mais eficaz de saber se o modelo não foi enviesado por algum atributo que foi deixado nos dados gerados.

Dificuldades

Tive dificuldades para achar fontes que explicassem melhor como funciona o fluxo de aplicação do Omniverse replicator.

Conclusões

A principal conclusão desse blog é que o omniverse replicator é uma excelente ferramenta para auxiliar na criação de dados sintéticos em 3D para modelos multimodais, pois ajuda a lidar com os principais desafios que normalmente são encontrados nesses cenários, mas não invalida a importância de utilizar outras ferramentas como o Blender para a criação de objetos e texturas, que ainda se mostram essenciais.

Referencias

Fontes do Card:

[NVIDIA Isaac Sim On Omniverse – Synthetic Data for Perception Model Training](#)

[Synthetic Data Generation with NVIDIA and Roboflow](#)

Fontes Extras:

[Slides da apresentação da Nvidia](#)

[Replicator](#)

[Construindo Pipelines de Geração de Dados Sintéticos Escaláveis para Percepção de IA com Databricks e NVIDIA Omniverse](#)

[Imagem retirada do blog macnica](#)